FASE 3 · PROYECTO 06 · EBPF

pktwatch

Observabilidad de red con eBPF:
tráfico TCP/UDP por proceso, en tiempo real

Versión 1.0 · Roadmap Sistemas Linux

Resumen

Vas a construir una herramienta tipo `top` que muestra, para cada proceso del sistema, cuántos datos y cuántos paquetes TCP y UDP envía. La lógica de captura corre **dentro del kernel** como programas eBPF verificados; la presentación, en un programa de user space con libbpf. Al terminar sabrás escribir, cargar y depurar programas eBPF portables (CO-RE), la tecnología que hay detrás de Cilium, Falco o Pixie.

CAPÍTULO I

Introducción

En `ringdev` metiste código arbitrario en el kernel: un fallo y adiós sistema. eBPF propone otro trato: escribes programas pequeños en C restringido, se compilan a bytecode BPF y, antes de ejecutarse, un **verificador** demuestra que terminan, que no acceden a memoria inválida y que solo usan helpers permitidos. Después, un compilador JIT los convierte en código nativo y se enganchan a puntos del kernel: funciones (kprobes, fentry), tracepoints, la pila de red…

Los programas eBPF no tienen estado propio entre ejecuciones; comparten datos con user space mediante **mapas** (tablas hash, arrays, ring buffers). Y gracias a BTF y CO-RE (*Compile Once – Run Everywhere*), el mismo binario funciona en kernels con distintas estructuras internas.

CAPÍTULO II

Instrucciones generales

- VM con kernel ≥ 5.15 con BTF (`ls /sys/kernel/btf/vmlinux` debe existir). Debian 12 o Ubuntu 22.04+ sirven.
- Toolchain: `clang`, `libbpf-dev`, `bpftool`. Genera `vmlinux.h` con `bpftool btf dump file /sys/kernel/btf/vmlinux format c` y usa *skeletons* (`bpftool gen skeleton`).
- Partes en C: el programa BPF (`*.bpf.c`) y el cargador en user space (`*.c`) con libbpf. Prohibido usar BCC o bpftrace en la entrega (sí puedes usarlos para explorar).
- Los accesos a estructuras del kernel se hacen con `BPF_CORE_READ` para que el binario sea portable.

- Se ejecuta como root (o con `CAP_BPF` y `CAP_PERFMON`). Al salir con Ctrl+C, todo se desengancha y se libera.
- Inspírate en la estructura de `libbpf-bootstrap`, pero el código debe ser tuyo.

CAPÍTULO III

Ejercicios

Ejercicio	00: execsnoop
Entrega	`ex00/`
Permitido	tracepoints, `BPF_MAP_TYPE_RINGBUF`, `bpf_get_current_pid_tgid`, `bpf_get_current_comm`, `bpf_probe_read_user_str`

Tu «hola mundo» de eBPF: muestra en tiempo real cada `execve` del sistema, con PID, PPID, nombre del proceso que lo ejecuta y ruta del binario. Engánchate al tracepoint `syscalls/sys_enter_execve` y envía eventos a user space por un ring buffer.

```
$ sudo ./execsnoop
PID     PPID    COMM            FILENAME
48211   1204    bash            /usr/bin/ls
48212   1204    bash            /usr/bin/git
48213   48212   git             /usr/lib/git-core/git-remote-https
```

PISTA

`bpf_get_current_pid_tgid()` devuelve un `u64`: la mitad alta es el **TGID**, lo que user space llama PID; la mitad baja es el ID del hilo, lo que el kernel llama PID. Confundirlos es el error número uno.

Ejercicio	01: pktwatch · bytes por proceso
Entrega	`pktwatch/`
Permitido	fentry/fexit o kprobes, `BPF_MAP_TYPE_PERCPU_HASH`, variables `const volatile` en `.rodata`

La parte central. Engánchate a las funciones de envío de la pila de red y acumula, por proceso y protocolo, el número de llamadas de envío y los bytes **realmente** enviados (el valor de retorno, no el tamaño pedido):

- TCP: `tcp_sendmsg` (cubre IPv4 e IPv6).
- UDP: `udp_sendmsg` **y** `udpv6_sendmsg`. Si olvidas la segunda, no verás nada del tráfico DNS sobre IPv6.
- Clave del mapa: `{tgid, protocolo}`. Valor: `{llamadas, bytes}`. Usa un mapa *per-CPU* para evitar contención entre CPU y suma los valores en user space.
- User space refresca cada segundo una tabla ordenada por tasa de bytes, con el nombre del proceso:

```
$ sudo ./pktwatch
PID     COMM            PROTO   CALLS/s   KiB/s      TOTAL
3120    firefox         TCP         412    1834.2    211.4 MiB
882     systemd-resolve UDP          36       2.1      1.3 MiB
51001   curl            TCP          97     955.7     40.0 MiB
```

- Opción `-p PID` para filtrar un proceso. El filtro se aplica **en el kernel**, mediante una variable global `const volatile` que se fija antes de cargar el programa, no filtrando en user space.
- Opción `-i SEG` para el intervalo de refresco y `-n N` para mostrar solo los N primeros.
- Los procesos que terminan desaparecen de la tabla: decide cómo detectarlo (desde el kernel o desde user space) y justifícalo en el README.

Ejercicio	02: pktwatch · paquetes
Entrega	`pktwatch/` (opción `--packets`)
Permitido	tracepoints de red (`net:net_dev_queue` u otro), mapas hash, `BPF_CORE_READ`

Ahora cuenta **paquetes** reales que salen por la interfaz, atribuidos a su proceso. Esto es mucho más difícil de lo que parece, y entender por qué es el objetivo del ejercicio:

- Cuando un paquete llega a la interfaz de red, a menudo ya no estás en el contexto del proceso que lo envió (puede salir desde una softirq, un temporizador de retransmisión o un ACK). `bpf_get_current_pid_tgid` te devolverá un proceso equivocado.
- Solución típica: en el momento del `sendmsg` (contexto correcto) asocia el socket (`struct sock *`) a su proceso en un mapa, y en el punto de salida del paquete obtén el socket desde el `sk_buff` y consulta ese mapa.
- Piensa cuándo y cómo limpiar esa asociación al cerrarse el socket. Documenta tu decisión.

En `NOTES.md`, sirve un fichero grande desde la VM o usa `iperf3` y compara bytes, llamadas y paquetes. Explica las discrepancias: TSO/GSO (un `sk_buff` puede convertirse en muchos paquetes en el cable), ACKs puros, retransmisiones y cabeceras.

Ejercicio	03: verifier diary
Entrega	`VERIFIER.md`
Permitido	`bpftool prog`, el log del verificador

Provoca deliberadamente al menos tres rechazos del verificador: desreferenciar el resultado de `bpf_map_lookup_elem` sin comprobar `NULL`, un bucle sin cota demostrable y una lectura fuera de los límites de la pila. Para cada uno, incluye el fragmento del log, explica qué razonamiento del verificador falla y cómo lo corregiste.

CAPÍTULO IV

Parte bonus

- Dirección de recepción (RX): bytes recibidos por proceso. Investiga qué funciones de la pila TCP y UDP te dan una atribución fiable.
- Agregación por cgroup (`bpf_get_current_cgroup_id`) para ver el tráfico por contenedor Docker o servicio systemd.
- Mide el sobrecoste de tus programas: activa `kernel.bpf_stats_enabled` y usa `bpftool prog show` (`run_time_ns`, `run_cnt`). Compara fentry frente a kprobe.
- Exporta las métricas en el formato de texto de Prometheus por HTTP. Si haces el proyecto 07, puedes servirlas desde `rhttpd`.
- Usa `BPF_MAP_TYPE_SK_STORAGE` para la asociación socket → proceso, si tu kernel lo permite desde tus programas de tracing, y compáralo con tu mapa hash.

CAPÍTULO V

Evaluación

Prueba	Esperado
`execsnoop` + abrir una terminal y lanzar comandos	Todos los `execve` aparecen con datos correctos.
`python3 -m http.server` en la VM y descarga de un fichero de 100 MiB desde fuera	`python3` aparece arriba con unos 100 MiB enviados por TCP.
`iperf3 -c` (TCP) y `iperf3 -u -c` (UDP) contra otra máquina	Bytes enviados que coinciden con iperf3 dentro de un margen razonable.
`dig @::1 example.com` con resolver IPv6	Aparece tráfico UDP.
`pktwatch -p PID`	Solo ese proceso; el filtro está en el programa BPF (comprobable en el código).
Ctrl+C y `bpftool prog list`	No queda ningún programa ni mapa de `pktwatch` cargado.
Mismo binario en dos kernels distintos (p. ej. Debian 12 y Ubuntu 24.04)	Funciona sin recompilar (CO-RE).
`NOTES.md` y `VERIFIER.md`	El evaluado explica el problema de atribución de paquetes y los rechazos del verificador.